

Kubernetes and Cloud Native Associate - KCNA

Container Orchestration Security

Image Security

In this lesson, we explore best practices for securing container images. We start with the fundamentals of image naming, then move to using secure image repositories, and finally discuss how to configure your pods to pull images from private registries.

Throughout this lesson, you will find practical examples that deploy various applications, including web apps, databases, and Redis caches. Let's begin by examining a simple pod definition file.

For instance, consider the following pod configuration:

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
spec:
  containers:
  - name:
```

In this example, the configuration uses the Nginx image to deploy a container. But what exactly is this image, and where does it come from?

When you specify an image like "nginx", you are following Docker's image naming convention. In this case, "nginx" refers to "library/nginx" because, without a user or account name, Docker defaults to the "library" account. This account hosts Docker's official images, maintained by a dedicated team to ensure best practices.

If you create your own account and publish images, you follow the same naming pattern. For example, your image might be stored under "yourname/imagename" instead of "library/imagename".

Since our configuration does not specify a location, the image is pulled from Docker's default registry, Docker Hub, accessible via the DNS name docker.io. In practice, when you reference the image as:

```
image: library/nginx
```

It is actually stored and pulled from:

```
image: docker.io/library/nginx
```

Whenever you create or update an image, you push it to the registry, and each time your application is deployed, the image is pulled from that registry.

Public and Private Registries

Several popular registries are available:

- **Docker Hub:** The default registry where official and community images are stored.
- **Google Container Registry (GCR):** Located at `gcr.io`, it hosts many Kubernetes-related images, including those used for end-to-end cluster testing. These images are publicly accessible.
- **Private Registries:** For in-house applications that should remain confidential, it is best to host your images in a private registry. Many cloud service providers,

including AWS, Azure, and GCP, offer private registries by default.



Tip

Restrict access to your repository using credentials, regardless of whether you're using Docker Hub, GCR, or an internal solution.

Using Private Images with Docker

To run a container from a private image, you must log in to the private registry with the Docker CLI. For example:

```
docker login private-registry.io
Login with your Docker ID to push and pull images from Docker Hub. If you don't h
Username: registry-user
Password:
WARNING! Your password will be stored unencrypted in /home/vagrant/.docker/config
Login Succeeded
```

After logging in, you can run your application using the private image:

```
docker run private-registry.io/apps/internal-app
```

Configuring Kubernetes to Use Private Registries

In Kubernetes, you need to provide the necessary credentials to allow pods to pull images from a private registry. This is achieved by creating a secret object that stores your Docker registry credentials.

First, create the secret using the following command:

```
kubectl create secret docker-registry regcred \  
  --docker-server=private-registry.io \  
  --docker-username=registry-user \  
  --docker-password=registry-password \  
  --docker-email=registry-user@org.com
```

Then, update your pod definition to reference this secret under the `imagePullSecrets` section:

```
apiVersion: v1  
kind: Pod  
metadata:  
  name: nginx-pod  
spec:  
  containers:  
    - name: nginx  
      image: private-registry.io/apps/internal-app  
  imagePullSecrets:  
    - name: regcred
```

When the pod is created, Kubernetes uses the credentials stored in the secret to authenticate with the private registry and pull the required image.



Key Information

Ensure that your secret's name in the pod definition matches the name you provided when creating the secret.

This concludes our lesson on image security. By following these steps, you can securely deploy container images from both public and private registries in your Kubernetes clusters.

For further reading, consider checking out:

- [Kubernetes Documentation](#)
- [Docker Hub](#)
- [Google Container Registry](#)
- **Kubernetes Basics**



Beta



Kubernetes and Cloud Native Associate - KCNA

watch video content

Previous

[Service Accounts](#)

Next

[Security Contexts](#)